HOCHSCHULE
MITTWEIDA
University of Applied Sciences

BACHELOR THESIS

Mr.
Georgii Burdin

Detecting Model Drift in LLM-as-a-Service APIs: A Benchmarking Framework for Campaign-Relevance Classification

Mittweida, April 2026

Faculty of **Applied Computer Sciences and Biosciences**

BACHELOR THESIS

Detecting Model Drift in LLM-as-a-Service APIs: A Benchmarking Framework for Campaign-Relevance Classification

Author:
Georgii Burdin

Course of Study:
B.Sc. Applied Mathematics

Seminar Group:
MA22w1-B

First Examiner:
Prof <Name> <First Examiner>

Second Examiner:
<Second Examiner>

Submission:
Mittweida, 01.04.2026

Defense/Evaluation:
Mittweida, 2026

Bibliographic Description

Burdin, Georgii:
Detecting Model Drift in LLM-as-a-Service APIs: A Benchmarking Framework for Campaign-Relevance Classification. – 2026. – 41 S.
Mittweida, Hochschule Mittweida – University of Applied Sciences, Faculty of Applied Computer Sciences and Biosciences, Bachelor Thesis, 2026.

Referat

1. Context
Gocomo uses API-based large language models (LLMs) for campaign-relevance classification on social-media posts.

2. Practical Problem
These API models change over time without full transparency, so the performance of a deployed classification pipeline may degrade.

3. Operational Constraint
Re-running a full benchmark frequently is too expensive.

4. Methodological Problem
Therefore, a framework is needed that can:

- represent the task consistently,
- monitor performance over time with a smaller but informative sample,
- detect drift statistically,
- support cost-aware model selection,
- and possibly recover performance through prompt optimisation.

Contents

List of Figures and Tables

List of Figures

List of Tables

1 Introduction

1.1 Context and Motivation

Industrial classification pipelines increasingly rely on Large Language Model (LLM) providers that expose their systems through application programming interfaces. In such settings, the deployed model can be viewed as a non-stationary black-box function $f_t(x)$ whose internal parameters and update schedule are controlled by the provider rather than by the user. As a result, the behaviour of the same API-accessible model may change over time even when the downstream task, the prompt, and the evaluation procedure remain fixed [1–3].

The practical motivation for this thesis arises from the author's two-year work experience at **Gocomo GmbH**. In the company's social-media analytics workflow, LLMs are used in batch mode to determine whether a post is relevant to a specified campaign, brand, event, or other marketing entity. This is naturally formulated as a binary classification task. Because the volume of incoming content is large, manual verification is not scalable, and automated classification becomes a necessary operational component.

However, the use of external LLM APIs creates a reliability problem. A model that performs well during an initial benchmark may later degrade due to silent provider-side updates. In a business-critical pipeline, such degradation must be detected early. At the same time, repeated full-scale benchmarking on a large labelled dataset is costly. This creates a need for a monitoring methodology that is statistically informative, operationally feasible, and economically justified.

1.2 Problem Statement

This thesis addresses the problem of how to evaluate and monitor API-based LLM classifiers in a production environment where model behaviour may change over time. The central challenge is that the user does not control the underlying model updates, yet must still make reliable deployment decisions based on observed task performance.

More specifically, the problem consists of four interconnected parts:

1. The social-media relevance detection task must be formalised in a way that permits consistent evaluation across models, prompts, and time points.
2. Because repeated evaluation on the full reference benchmark is economically expensive, a smaller monitoring subset must be constructed without losing the ability to detect practically relevant performance changes.
3. A statistical rule is required to distinguish variability in performance from genuine model drift.
4. Once multiple candidate models are monitored, deployment decisions must account not only for predictive performance but also for inference cost.

In addition, if a deployed model exhibits degraded performance, it is natural to ask whether this degradation can be mitigated without immediately replacing the model. This motivates the investigation of prompt optimisation as an adaptive mechanism within the monitoring framework.

Accordingly, the thesis does not study model drift merely as an observational phenomenon. Its objective is to develop a practically usable framework for benchmark design, drift-sensitive monitoring, cost-aware model selection, and post-drift adaptation for binary classification with black-box LLM APIs.

1.3 Research Questions

The problem stated above is investigated through the following research questions:

RQ1: How can the campaign-relevance detection task be formally represented as a binary classification problem for non-stationary black-box LLM APIs?
RQ2: How can a reduced monitoring subset be derived from a larger reference benchmark so that routine evaluation remains substantially cheaper while preserving sensitivity to model drift?
RQ3: Which statistical criteria and alerting rules are appropriate for detecting practically significant performance degradation over time?
RQ4: How should candidate models be compared and selected when deployment decisions depend on both predictive quality and inference cost?
RQ5: To what extent can prompt optimisation improve or recover task performance after drift has been detected, and how can such optimisation be integrated into the monitoring loop?

1.4 Task Definition

1.4.1 What do we do?

At a high level, the production pipeline takes each incoming social-media post and asks an LLM whether the content is relevant to a given marketing campaign. The model sees a structured description of the campaign (brands, slogans, product lines) together with the post's text, hashtags, and selected video frames, and must decide if at least one of the campaign entities is mentioned, visually present (e.g. logo), or otherwise clearly implied. The LLM returns a binary decision for every post in the batch, which is then used downstream for filtering and reporting.

1.4.2 Example LLM Request Payload

During evaluation, each social-media post is converted into a structured request sent to the LLM provider API. The request consists of a system prompt describing the classification task together with user inputs containing the extracted content of the post (text, metadata, and video frames).

A simplified example payload is shown below.

NB!: MAKE A VISUALISATION IN FIGMA OF THE PAYLOAD WITH A IMAGE → STRUCTURED OUTPUT, it is literally a map function

```
[
  {
    "role": "system",
    "content": "Classify whether the post is relevant to the campaign
    'Douglas_Beauty Day 2025'. Return TRUE or FALSE and explain the reasoning."
  },
  {
    "role": "user",
    "content": "Transcription: ..."
  },
  {
    "role": "user",
    "content": "Hashtags: []"
  },
  {
    "role": "user",
    "content": [
      {
        "type": "input_text",
        "text": "Video frames extracted from the post"
      },
      {
        "type": "input_image",
        "image_url": "https://.../frame1.jpg"
      },
      {
        "type": "input_image",
        "image_url": "https://.../frame2.jpg"
      }
    ]
  }
]
```

The system prompt defines the classification criteria and the list of campaign brands. The user messages contain the multimodal content of the post, including captions, transcriptions, hashtags, and selected video frames. Results are restricted to binary response of is this post relevant or not to the requested topic

To evaluate model drift rigorously, we must first define the classification task within a probabilistic framework.

2 Data and Evaluation Setup

2.1 Formal Problem Definition

To evaluate model drift rigorously, we must first define the classification task within a probabilistic framework. Let $\mathcal{X} \subset \mathbb{R}^d$ denote the high-dimensional input space of social-media posts (comprising text, metadata, and visual features) and $\mathcal{Y} = \{0, 1\}$ the binary label space, where 1 indicates relevance to a campaign.

In the context of LLM-as-a-Service, the model is a non-stationary black box. We define the classifier as a function $f(x; \pi, \theta_t) \to \hat{y} \in \{0, 1\}$, where π is a fixed prompt and θ_t represents the hidden, time-varying model weights.

Under the zero-one loss function $\mathcal{L}(\hat{y}, y) = \mathbb{I}(\hat{y} \neq y)$, the **Expected Risk** (or true error rate) at time t is:

$$R_t(f) = \mathbb{E}_{(x,y)\sim P_t}[\mathbb{I}(f(x; \pi, \theta_t) \neq y)] \tag{2.1}$$

Since the true data-generating distribution P_t is unknown, we must estimate this risk empirically.

2.2 The Reference Benchmark vs. Operational Constraints

Let $\mathcal{D}_N$ be the reference benchmark consisting of $N = 1200$ manually labelled examples, establishing the ground truth. Evaluating all candidate models against $\mathcal{D}_N$ yields a highly precise empirical risk $\hat{R}_N$.

However, running inference on $N = 1200$ instances weekly across multiple candidate models is economically prohibitive. We require a monitoring subset $\mathcal{D}_n \subset \mathcal{D}_N$ of size $n \ll N$. The central mathematical challenge is to select $\mathcal{D}_n$ such that it remains highly sensitive to temporal shifts in the parameters θ_t, despite the significantly reduced sample size.

2.3 Informative Sampling via Model Disagreement

A simple random sample (SRS) of size n would yield an unbiased estimator of R_t. However, the vast majority of social-media posts are "easy" instances located far from the model's decision boundary, where the conditional probability $P(Y = 1|X) \approx 1$ or 0. These instances exhibit low prediction variance and are highly robust to small parameter shifts in θ_t.

To maximize the sensitivity of our monitoring subset, we abandon uniform sampling in favor of **Uncertainty Sampling**. We construct $\mathcal{D}_n$ exclusively from instances where an initial ensemble of distinct models $\mathcal{M} = \{f_1, f_2, \ldots, f_K\}$ exhibited disagreement:

$$\mathcal{D}_n = \{x_i \in \mathcal{D}_N \mid \exists f_j, f_k \in \mathcal{M} : \hat{y}_{i,j} \neq \hat{y}_{i,k}\} \tag{2.2}$$

To understand why this maximizes detection power, we must formalize the geometry of the classifier's decision space and prove that disagreement isolates instances of maximal statistical variance.

2.3.1 Geometry of the Decision Boundary

In modern representation learning, it is standard to assume the *Manifold Hypothesis*: the tenet that nominally high-dimensional data are in fact concentrated near a lower-dimensional submanifold [4, 5]. This assumption serves as a primary inductive bias for deep learning architectures [6], as it allows models to circumvent the curse of dimensionality by focusing on the intrinsic geometric structure of the data rather than the ambient space.

Assumption 1 (Data Manifold)
Let the input space $\mathcal{X} \subset \mathbb{R}^d$ be a smooth, compact Riemannian manifold.

For any ensemble $\mathcal{M}$, let $\bar{p} : \mathcal{X} \to [0, 1]$ be the continuous consensus probability function, where $\bar{p}(x)$ represents the ensemble's expected probability that $Y = 1$.

Definition 2.1 (Consensus Decision Boundary)
The decision boundary $\mathcal{B}$ of the ensemble is the pre-image of the classification threshold:

$$\mathcal{B} = \bar{p}^{-1}(0.5) = \{x \in \mathcal{X} \mid \bar{p}(x) = 0.5\} \tag{2.3}$$

Proposition 1
*If 0.5 is a regular value of the function $\bar{p}$ (i.e., the gradient $\nabla \bar{p}(x) \neq 0$ for all $x \in \mathcal{B}$), then by the **Regular Level Set Theorem**, the decision boundary $\mathcal{B}$ is a smooth submanifold of $\mathcal{X}$ of codimension 1.*

This establishes $\mathcal{B}$ as a formal geometric hypersurface dividing the data manifold $\mathcal{X}$ into two distinct classification regions.

2.3.2 Proof of Maximal Uncertainty via Disagreement

We now prove that sampling instances where models disagree mathematically confines the sample to a tight neighborhood around the submanifold $\mathcal{B}$.

Assumption 2 (Bounded Ensemble Divergence)
Assume the models in $\mathcal{M}$ represent competent classifiers approximating the same underlying logic, such that their individual scoring functions $p_m(x)$ diverge from the consensus $\bar{p}(x)$ by no more than ϵ:

$$\sup_{x \in \mathcal{X}} |p_m(x) - \bar{p}(x)| \leq \epsilon \quad \textit{for all } m \in \{1, \ldots, K\} \tag{2.4}$$

Proposition 2 (Disagreement Bounds Distance to $\mathcal{B}$)
If there exists a disagreement within the ensemble $\mathcal{M}$ for a given instance x (i.e., $p_j(x) < 0.5$ and $p_k(x) \geq 0.5$), then the consensus probability $\bar{p}(x)$ is strictly bounded within an ϵ-neighborhood of 0.5.

Proof: Applying the bounded divergence assumption to the disagreeing models:

$$\bar{p}(x) - \epsilon \leq p_j(x) < 0.5 \implies \bar{p}(x) < 0.5 + \epsilon \tag{2.5}$$

$$\bar{p}(x) + \epsilon \geq p_k(x) \geq 0.5 \implies \bar{p}(x) \geq 0.5 - \epsilon \tag{2.6}$$

Thus, it strictly follows that $\bar{p}(x) \in [0.5 - \epsilon,\ 0.5 + \epsilon]$. □

By geometrically isolating instances where models produce conflicting predictions, we are effectively sampling directly from the ϵ-tube surrounding the submanifold $\mathcal{B}$. At the theoretical limit ($\epsilon \to 0$), the binary entropy $H(Y|X)$ reaches its global maximum of 1 bit. The model is maximally uncertain, meaning the expected accuracy approaches random guessing ($A \approx 0.5$).

This geometric isolation formalizes why disagreement sampling forces the evaluation subset into the **Maximum Variance** state for a Bernoulli process, which serves as the foundational assumption for determining the required sample size.

2.4 Statistical Derivation of the Sample Size

The decision to sample from the ϵ-neighborhood of the decision boundary directly dictates the required sample size n. By forcing the expected accuracy toward $A = 0.5$, we deliberately subject our evaluation to the **Maximum Entropy** state. For a Bernoulli process, the variance $\sigma^2 = A(1 - A)$ reaches its absolute maximum of 0.25 exactly at $A = 0.5$.

NB! show disagreement of annotators

To determine n under this worst-case variance, we must define the **Minimum Detectable Effect (MDE)** Δ. In our production environment, human inter-annotator disagreement establishes an irreducible noise floor of $\epsilon_{noise} \approx 0.05$ (5%). Any model drift smaller than ϵ_{noise} is statistically indistinguishable from background label noise. Thus, our monitoring system must have a margin of error strictly bounded by $\Delta = 0.05$.

We formalize this requirement using the margin of error for a one-sided hypothesis test (detecting degradation) at a significance level of $\alpha = 0.05$. Let $Z_{1-\alpha} \approx 1.645$ be the critical value from the standard normal distribution. The required sample size n must satisfy:

$$Z_{1-\alpha}\sqrt{\frac{A(1-A)}{n}} \leq \Delta \tag{2.7}$$

Substituting the maximum variance assumption $A(1-A) = 0.25$ and our noise floor $\Delta = 0.05$, we obtain:

$$1.645\sqrt{\frac{0.25}{n}} \leq 0.05 \tag{2.8}$$

$$\frac{0.8225}{\sqrt{n}} \leq 0.05 \implies \sqrt{n} \geq 16.45 \tag{2.9}$$

$$n \geq 270.6 \tag{2.10}$$

Voluntarly, the theoretically required sample size of $n \geq 271$ is rounded up to $n = 300$. This calculation proves that 300 is the strict mathematical minimum required to detect a 5% model drift under the maximum variance conditions induced by our informative sampling strategy.

2.5 The Role of Selection Bias and Importance Weighting

Because $\mathcal{D}_{300}$ intentionally over-represents the manifold's decision boundary, the empirical accuracy calculated directly on this subset, $\hat{A}_n$, is a **biased estimator** of the global population accuracy A_N. Formally, $\mathbb{E}[\hat{A}_n] < A_N$, as the subset is deliberately composed of instances with high epistemic uncertainty.

In standard statistical evaluation, selection bias is viewed as a flaw to be corrected. In the context of our drift monitoring framework, however, this bias is an operational requirement. We track the biased metric $\hat{A}_n$ because it isolates the exact region of the input space where temporal parameter shifts ($\Delta\theta_t$) manifest most strongly, thereby maximizing the statistical power of the alerting rule.

Nevertheless, it is necessary to mathematically guarantee that this heavily biased subset remains formally linked to the underlying data-generating distribution, rather than being an arbitrary or disconnected sample. This rigorous link is established through the theory of **Importance Sampling** and the **Change of Measure**.

2.5.1 The Horvitz–Thompson Link

Let $S_i \in \{0, 1\}$ be an indicator variable for the inclusion of the i-th reference instance in the monitoring subset, with probability $q_i = P(S_i = 1 \mid \text{disagreement})$. The global accuracy A_N can be theoretically recovered from the biased sample using the **Horvitz–Thompson (HT) estimator**:

$$\hat{A}_{HT} = \frac{1}{N}\sum_{i=1}^{N} \frac{S_i \cdot \mathbb{I}(\hat{y}_i = y_i)}{q_i} = \frac{1}{N}\sum_{i \in \mathcal{D}_n} \frac{\mathbb{I}(\hat{y}_i = y_i)}{q_i} \tag{2.11}$$

Here, the inverse probability $1/q_i$ acts as an importance weight (or Radon–Nikodym derivative) that maps the biased sampling measure back to the uniform population measure.

Proof of Unbiasedness: To prove that $\hat{A}_{HT}$ correctly recovers the global accuracy, we take its expectation over the sampling design:

$$\mathbb{E}[\hat{A}_{HT}] = \mathbb{E}\left[\frac{1}{N}\sum_{i=1}^{N}\frac{S_i \cdot \mathbb{I}(\hat{y}_i = y_i)}{q_i}\right] \tag{2.12}$$

By the linearity of expectation, and noting that the true labels and model predictions at a fixed time t are deterministic with respect to the sampling mechanism:

$$\mathbb{E}[\hat{A}_{HT}] = \frac{1}{N}\sum_{i=1}^{N}\frac{\mathbb{I}(\hat{y}_i = y_i)}{q_i}\mathbb{E}[S_i] \tag{2.13}$$

Since S_i is a Bernoulli random variable, its expected value is exactly its probability of occurrence, $\mathbb{E}[S_i] = q_i$. Substituting this yields:

$$\mathbb{E}[\hat{A}_{HT}] = \frac{1}{N}\sum_{i=1}^{N}\frac{\mathbb{I}(\hat{y}_i = y_i)}{q_i} \cdot q_i = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}(\hat{y}_i = y_i) = A_N \tag{2.14}$$

□

2.5.2 Operational Utility: Signal Amplification

The proof above confirms that $\mathcal{D}_{300}$ is a statistically valid basis for inference, as it remains anchored to the reference population through the HT estimator. However, within the operational monitoring loop, we deliberately report the raw empirical accuracy $\hat{A}_n$ rather than the unbiased estimate $\hat{A}_{\mathrm{HT}}$.

The rationale for this choice is rooted in **Signal-to-Noise Ratio (SNR) optimization**. Applying the importance weights $1/q_i$ would mathematically suppress the frequency of errors occurring at the decision boundary, effectively "smoothing" the data to reconstruct the global mean. While this is desirable for population estimation, it is counter-productive for **anomaly detection**. By utilizing the raw biased metric, we maintain the magnification of errors where the model is most unstable. Thus, $\hat{A}_n$ functions as a high-gain monitoring signal that provides a leading indicator of model drift, while the HT-framework provides the necessary theoretical guarantee that this signal is a rigorous and mathematically reversible projection of the global performance.

3 Drift Monitoring and Model Selection Methodology

3.1 Formalisation of Temporal Evaluation

Let $t \in \{1, \ldots, T\}$ represent discrete evaluation intervals, for example weekly batch runs. At each time point t, a deployed model m is evaluated on the same fixed monitoring subset $\mathcal{D}_{300} = \{(x_i, y_i)\}_{i=1}^{300}$. The key idea of this setup is that temporal changes in model behaviour should be measured on a constant reference set rather than on newly sampled data, so that differences across time can be attributed to model behaviour rather than to changes in the evaluation sample.

For each instance $x_i \in \mathcal{D}_{300}$, let

$$\hat{y}_{i,t} \in \{0, 1\} \tag{3.1}$$

denote the model prediction at time t. Since the true label y_i is fixed, the correctness of the prediction at time t can be represented by the binary indicator

$$E_{i,t} = \mathbb{I}(\hat{y}_{i,t} = y_i), \tag{3.2}$$

where $\mathbb{I}(\cdot)$ is the indicator function. Thus, $E_{i,t} = 1$ means that the model classified instance i correctly at time t, whereas $E_{i,t} = 0$ means that it made an error.

Using these indicators, the empirical accuracy of the model at time t on the monitoring subset is

$$\hat{A}_t = \frac{1}{300} \sum_{i=1}^{300} E_{i,t}. \tag{3.3}$$

This quantity is the fraction of correctly classified instances in the monitoring run. It provides a natural summary of aggregate predictive performance, but on its own it is not sufficient for rigorous drift detection.

The reason is that the sequence $(\hat{A}_t)_{t=1}^{T}$ is obtained by repeatedly evaluating the *same* instances. Therefore, the empirical accuracies $\hat{A}_{t_1}$ and $\hat{A}_{t_2}$ are not independent sample proportions. A standard two-sample comparison such as the two-proportion Z-test would incorrectly treat the observations as independent and would therefore overestimate the sampling variance. In the present setting, the dependence structure must be taken into account explicitly. This motivates the use of a paired test based on per-instance changes in correctness.

3.2 Statistical Drift Detection via McNemar's Test

To formally detect temporal drift, it is more informative to analyse *which instances changed status* than to compare marginal accuracies alone. In other words, the relevant signal is not simply whether the accuracy has changed, but whether instances that were previously correct have become incorrect, or vice versa.

3.2.1 The Contingency Table

For this purpose, the correctness indicators at two time points t_1 and t_2 are compared instance by instance. Each example in the monitoring subset falls into one of four categories:

- correct at both times,
- correct at t_1 but incorrect at t_2,
- incorrect at t_1 but correct at t_2,
- incorrect at both times.

Aggregating these outcomes across all $n = 300$ monitored instances yields the paired 2×2 contingency table for (t_1, t_2). The corresponding cell counts can be written directly inside the table as

Table 3.1: Paired 2×2 contingency table for correctness at times t_1 and t_2.

	$E_{i,t_2} = 1$	$E_{i,t_2} = 0$
$E_{i,t_1} = 1$	$a = \sum_{i=1}^{300} \mathbb{I}(E_{i,t_1} = 1,\ E_{i,t_2} = 1)$	$b = \sum_{i=1}^{300} \mathbb{I}(E_{i,t_1} = 1,\ E_{i,t_2} = 0)$
$E_{i,t_1} = 0$	$c = \sum_{i=1}^{300} \mathbb{I}(E_{i,t_1} = 0,\ E_{i,t_2} = 1)$	$d = \sum_{i=1}^{300} \mathbb{I}(E_{i,t_1} = 0,\ E_{i,t_2} = 0)$

These four counts partition the monitoring subset, so that

$$a + b + c + d = 300. \tag{3.4}$$

The interpretation of the cells is as follows:

- a: instances classified correctly at both time points,
- b: instances that were correct at t_1 but became incorrect at t_2,
- c: instances that were incorrect at t_1 but became correct at t_2,
- d: instances classified incorrectly at both time points.

From the perspective of drift detection, the concordant counts a and d are not informative about temporal change, because they correspond to cases where the model's behaviour did not change. The evidence for drift is therefore concentrated in the discordant counts b and c. In particular, b captures regressions, whereas c captures improvements.

This decomposition also clarifies the relation between paired comparisons and aggregate accuracy at the two time points. Since

$$\hat{A}_{t_1} = \frac{a+b}{300} \qquad \text{and} \qquad \hat{A}_{t_2} = \frac{a+c}{300}, \tag{3.5}$$

it follows that

$$\hat{A}_{t_2} - \hat{A}_{t_1} = \frac{c-b}{300}. \tag{3.6}$$

Hence, the change in accuracy is determined entirely by the imbalance between regressions and improvements. McNemar's test exploits exactly this structure.

3.2.2 The Test Statistic and Alerting Rule

The null hypothesis of McNemar's test states that the probability of a regression equals the probability of an improvement:

$$H_0 : p_b = p_c. \tag{3.7}$$

Under this null hypothesis, the model is equally likely to lose correctness on an instance as it is to gain correctness on another instance, which corresponds to the absence of systematic drift.

Since the operational concern is degradation, the relevant one-sided alternative is

$$H_a : p_b > p_c. \tag{3.8}$$

This means that the model breaks more previously correct cases than it repairs previously incorrect ones.

To test this hypothesis, McNemar's statistic with continuity correction is used:

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c}. \tag{3.9}$$

The numerator measures the imbalance between regressions and improvements, while the denominator normalises this imbalance by the total number of changed cases. The subtraction of 1 is the continuity correction, which is commonly used to make the large-sample chi-square approximation more conservative for finite samples.

The test statistic is approximately χ^2-distributed with one degree of freedom when the number of discordant pairs is sufficiently large. Operationally, the monitoring pipeline raises a **Statistical Drift Alert** at time t if both of the following conditions hold:

1. **Directionality:** $b > c$, meaning that the model has deteriorated on more instances than it has improved.
2. **Significance:** the corresponding p-value is below the chosen threshold $\alpha = 0.05$.

The directionality condition is important because statistical significance alone does not distinguish between degradation and improvement. By requiring both $b > c$ and $p < 0.05$, the alerting rule is aligned with the practical objective of detecting harmful drift rather than any change whatsoever.

3.2.3 Empirical Example from the Monitoring Runs

To make the above definitions concrete, Tables 3.2 and 3.3 show example McNemar contingency tables from two real monitoring runs on the $n = 300$ subset.

Table 3.2: McNemar contingency table for `gpt-5` between 8 March 2026 and 10 March 2026 on the 300-example monitoring subset.

	2026-03-10 correct	2026-03-10 wrong
2026-03-08 correct	291	0
2026-03-08 wrong	0	9

Here the discordant counts are $(b, c) = (0, 0)$, so there are no instances whose correctness changed between the two dates. McNemar's test is undefined in this degenerate case, but operationally the absence of discordant pairs already implies that no temporal drift is detectable on the monitoring subset. The corresponding dashboard view is shown earlier in Figure 5.4.

Table 3.3: McNemar contingency table for `gpt-5.1-mini` between 8 March 2026 and 10 March 2026 on the 300-example monitoring subset.

	2026-03-10 correct	2026-03-10 wrong
2026-03-08 correct	269	13
2026-03-08 wrong	7	11

In this second example, the discordant pairs are $(b, c) = (13, 7)$, yielding McNemar's chi-square statistic with continuity correction

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c} \approx 1.26$$

and an exact one-sided binomial p-value of approximately 0.26. Because $b > c$ but $p > 0.05$, the model exhibits a numerical decrease in accuracy but the evidence is insufficient to trigger a statistical drift alert under the chosen monitoring rule.

3.3 Cost-Aware Model Selection

Once performance has been measured, model selection must take into account not only predictive quality but also operational cost. In real deployments, a model that is slightly more accurate may still be unattractive if it is substantially more expensive to run at scale.

Let

$$A_t(m) \tag{3.10}$$

denote the empirical accuracy of model $m \in \mathcal{M}$ at time t, and let

$$C_t(m) \tag{3.11}$$

denote its average inference cost, measured in USD per 1 requests. The goal is to compare candidate models using a criterion that balances these two dimensions.

3.3.1 The Deployment Utility Function

To formalise this trade-off, define the deployment utility

$$U_t(m) = A_t(m) - \lambda \log\big(1 + C_t(m)\big), \tag{3.12}$$

where $\lambda \geq 0$ is a business-defined cost sensitivity parameter.

The first term, $A_t(m)$, rewards predictive performance. The second term penalises operational cost. The parameter λ controls how strongly cost influences the final decision:

- if $\lambda = 0$, then
$$U_t(m) = A_t(m), \tag{3.13}$$
so model selection is based purely on accuracy;
- if $\lambda > 0$, cost is explicitly incorporated into the deployment decision.

The logarithmic form of the penalty was chosen during a group workshop, where the value of λ was set to 10 to reflect the team's consensus on cost sensitivity in deployment scenarios. Unlike a linear penalty, the logarithmic option introduces diminishing marginal sensitivity to cost: a move from a very cheap to a moderately priced model is penalized more heavily than the same absolute cost increase at higher price levels, which better matches business intuition that value does not decrease linearly with API cost.

The corresponding dashboard controls are shown earlier in Figure 5.5. Figures 3.1–3.3 then illustrate the concrete per-model utilities and the behaviour of $U_t(m)$ as λ varies.

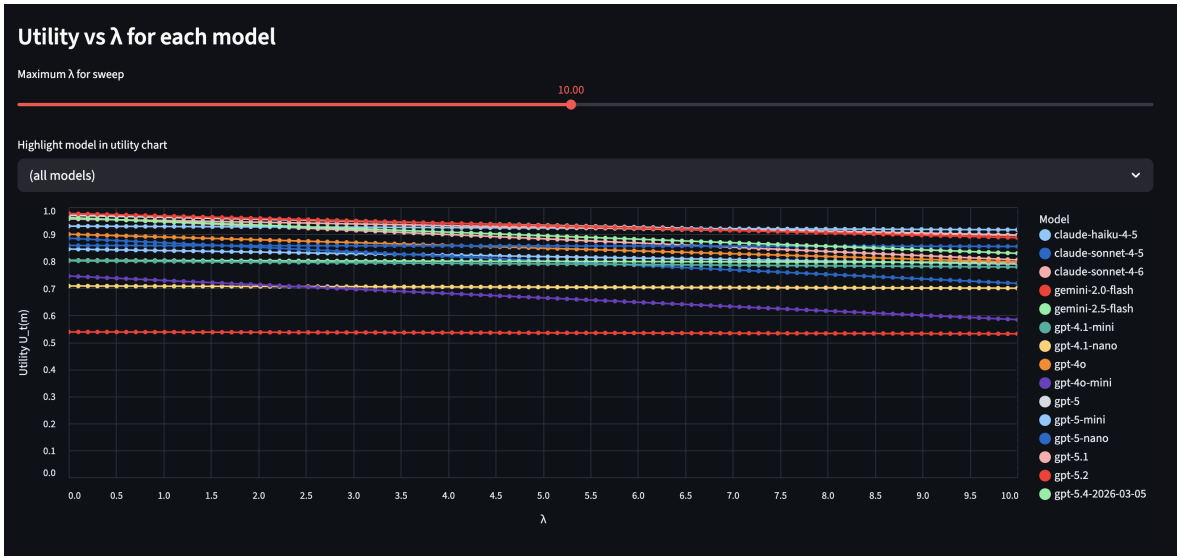


Figure 3.1: Per-model metrics and utility values for $\lambda = 10$ on the 300-example monitoring subset. The highlighted row shows the model selected by maximising $U_t(m)$, jointly accounting for accuracy and cost.

The use of $\log(1 + C_t(m))$ instead of $\log(C_t(m))$ also ensures that the expression is well-defined when the cost is close to zero.

3.3.2 Decision Logic for Pipeline Adaptation

The monitoring architecture follows a simple decision process after each batch evaluation:

1. **Monitor:** compute the paired drift statistic for the currently deployed model m_{curr} by comparing its correctness on $\mathcal{D}_{300}$ at time t against the chosen baseline time point.
2. **Detect:** if the McNemar test indicates significant degradation, that is, if $b > c$ and $p < 0.05$, register a drift event.

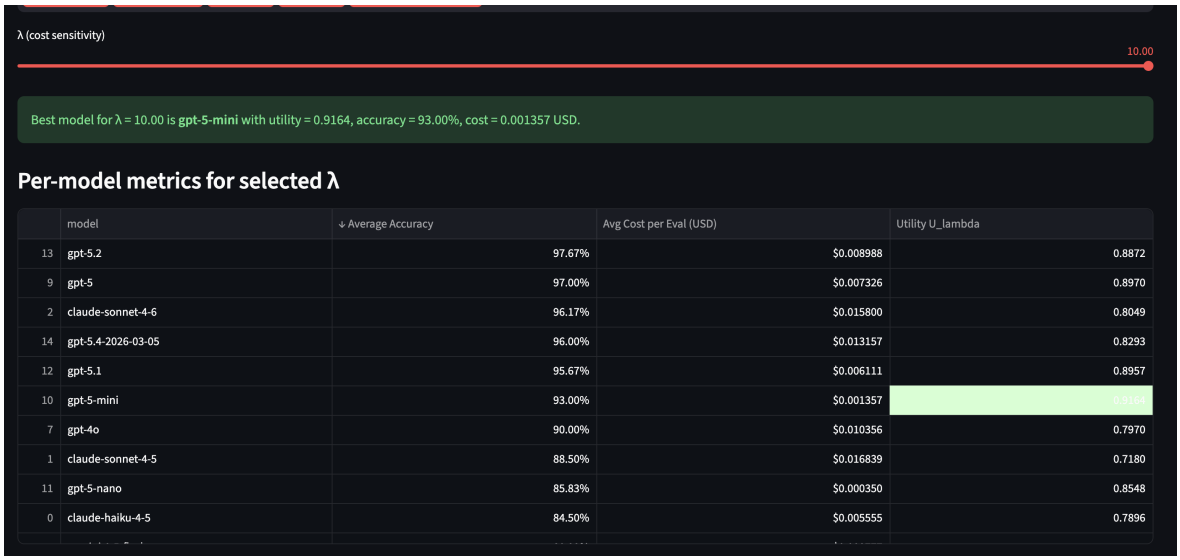


Figure 3.2: Utility $U_t(m)$ as a function of the cost-sensitivity parameter λ for all candidate models. The gradual flattening of the curves with increasing λ reflects the diminishing marginal penalty introduced by the $\log(1 + C_t(m))$ term.

3. **Act:** evaluate the deployment utility $U_t(m)$ for all available candidate models and select

$$m_t^\star \in \arg\max_{m \in \mathcal{M}} U_t(m). \tag{3.14}$$

Thus, the selected deployment model is the one that maximises the joint accuracy–cost criterion at time t. If no available fallback model yields a satisfactory utility level, the system initiates the prompt optimisation loop described in Chapter 4. In that case, adaptation is attempted first at the prompt level, without replacing the underlying model API.

Overall, this methodology separates the monitoring problem into two layers. First, McNemar's test determines whether a meaningful temporal degradation has occurred. Second, the utility function

$$U_t(m) \tag{3.15}$$

determines which model is most attractive from the deployment perspective once both predictive quality and cost are taken into account.

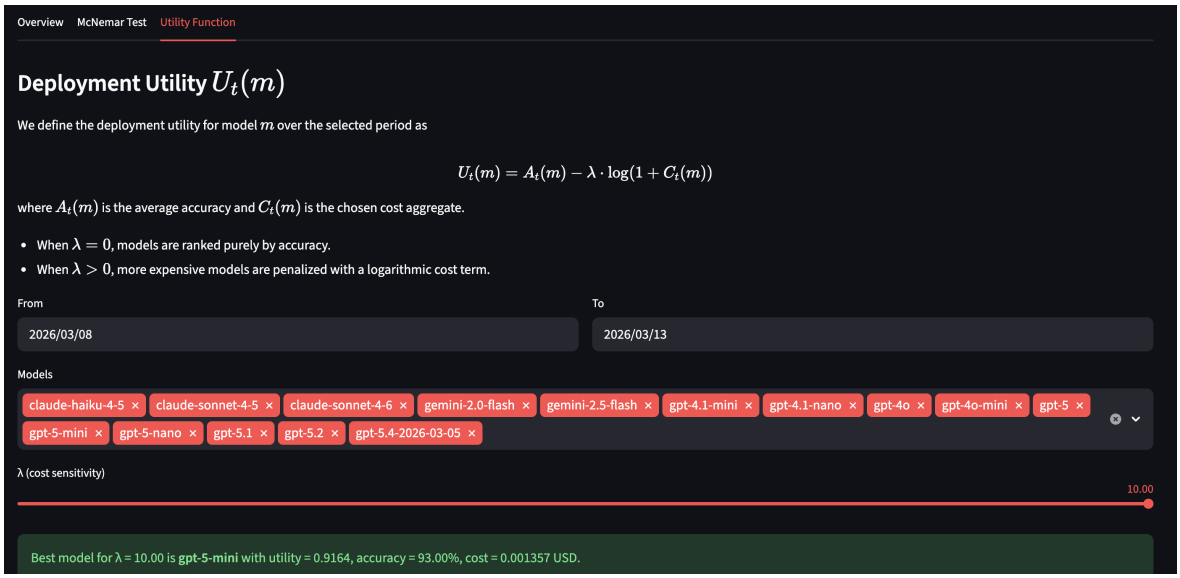


Figure 3.3: Same utility-vs-λ plot as in Figure 3.2, but with a single model highlighted to make the effect of increasing λ on its deployment attractiveness more visible.

4 Prompt Optimisation as an Adaptive Mechanism

4.1 Motivation for Prompt Adaptation

When the monitoring framework from Chapter 3 detects statistically significant degradation, the deployed system has changed its observable behaviour while the task definition remains fixed. In the present setting, this degradation cannot be addressed by fine-tuning the underlying model, because the LLM is accessed only through a provider API and fine-tuning is not economically reasonable. Thus, the only controllable part of the inference pipeline is the prompt.

Let the deployed classifier at time t be written as

$$f_t(x;\pi) = f(x;\pi,\theta_t), \tag{4.1}$$

where π is the prompt and θ_t denotes the unknown provider-controlled model state at time t. After drift, the hidden parameter state θ_t may have changed while π has remained fixed. Prompt optimisation therefore aims to find a modified prompt $\pi^\star$ such that the predictive quality of the classifier is restored or improved without changing the underlying model or incurring the higher cost of switching to a different LLM.

This makes prompt optimisation an attractive *adaptive mechanism* inside the monitoring loop. It allows the system to react to drift while preserving deployment continuity, controlling cost, and avoiding full model replacement unless adaptation fails.

4.2 General Formulation of Automatic Prompt Optimisation

Automatic Prompt Optimisation (APO) can be formalised as an optimisation problem over a discrete language space. Let $\mathcal{P}$ denote the set of admissible prompts, let $\mathcal{D}_{\text{opt}} = \{(x_i, y_i)\}_{i=1}^n$ denote the optimisation dataset, and let m_t denote the API model available at time t. For a prompt $\pi \in \mathcal{P}$, the empirical task score is

$$J_t(\pi) = \frac{1}{n}\sum_{i=1}^{n} \ell(f_t(x_i;\pi), y_i)\,, \tag{4.2}$$

where ℓ is a task-specific utility function. For binary classification, the natural choice is the correctness indicator

$$\ell(f_t(x_i;\pi), y_i) = \mathbb{I}(f_t(x_i;\pi) = y_i)\,, \tag{4.3}$$

so that $J_t(\pi)$ is simply the empirical accuracy of prompt π on the optimisation set.

The APO problem is then

$$\pi_t^\star \in \arg\max_{\pi\in\mathcal{P}} J_t(\pi). \tag{4.4}$$

In practice, this optimization problem cannot be solved exactly because the prompt space consists of discrete token sequences whose size grows combinatorially with sequence length. Consequently, most APO methods rely on approximate search procedures that iteratively generate, evaluate, and filter candidate prompts.

This problem is difficult for three reasons:

1. $\mathcal{P}$ is a *discrete combinatorial space* of natural-language strings rather than a continuous Euclidean parameter space.
2. The objective is typically *black-box*: only model outputs are observable, while gradients and internal activations are unavailable.
3. Evaluation is expensive because every candidate prompt requires repeated API calls on labelled examples.

A convenient abstract view of APO is as an iterative search process. Let $\mathcal{C}_k$ be the set of prompt candidates at iteration k, let $\mathcal{E}_k$ be the evaluation feedback obtained at that iteration, and let $\mathcal{U}$ denote the update rule. Then many APO methods can be written as

$$\mathcal{C}_0 = \mathrm{Init}(\mathcal{D}_{\mathrm{opt}}), \tag{4.5}$$

$$\mathcal{E}_k = \mathrm{Eval}(\mathcal{C}_k, \mathcal{D}_{\mathrm{opt}}), \tag{4.6}$$

$$\mathcal{C}_{k+1} = \mathcal{U}(\mathcal{C}_k, \mathcal{E}_k). \tag{4.7}$$

The final prompt is selected as

$$\pi^\star \in \arg\max_{\pi \in \mathcal{C}_K} J_t(\pi). \tag{4.8}$$

These equations describe APO as a repeated *generate–evaluate–update* procedure. The first step constructs an initial candidate set $\mathcal{C}_0$, for example from a manually written seed prompt or from several prompts generated from demonstrations. The second step evaluates the current candidates on $\mathcal{D}_{\mathrm{opt}}$ and produces feedback $\mathcal{E}_k$. In the simplest case, this feedback is just the set of validation scores,

$$\mathcal{E}_k = \{J_t(\pi) : \pi \in \mathcal{C}_k\}, \tag{4.9}$$

but it may also include richer information such as error cases, textual critiques, or execution traces. The third step applies the update rule $\mathcal{U}$, which generates the next candidate set from the current prompts and their feedback.

The final line,

$$\pi^\star \in \arg\max_{\pi \in \mathcal{C}_K} J_t(\pi), \tag{4.10}$$

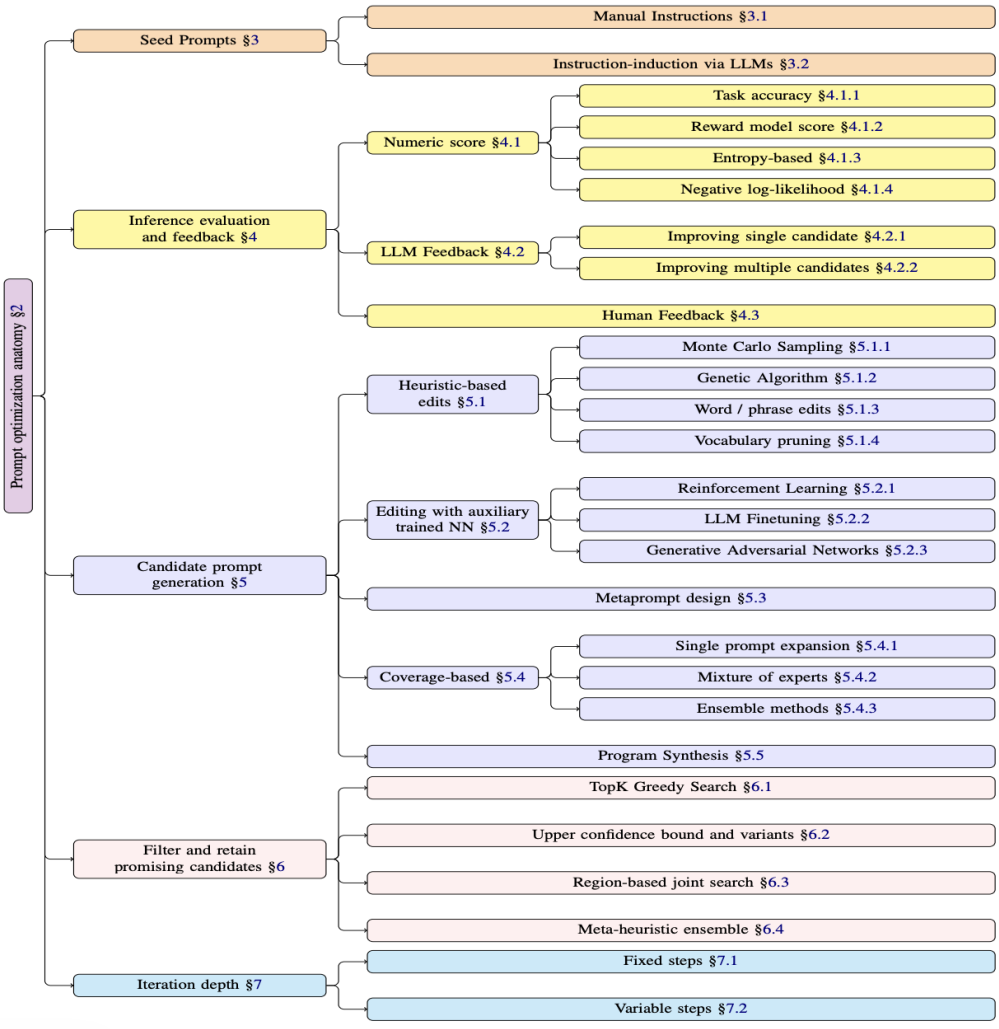
means that after K iterations the optimizer returns a prompt from the final candidate set with the highest empirical score. The symbol $\arg\max$ refers to the prompt attaining the maximum, not to the maximum score itself.

This abstraction is useful because different APO methods mainly differ in three places: how they initialize $\mathcal{C}_0$, what information is included in $\mathcal{E}_k$, and how the update operator $\mathcal{U}$ produces new candidates.

Seed prompt(s) $\rightarrow$ Candidate generation $\rightarrow$ Evaluation on $\mathcal{D}_{\text{opt}}$ $\rightarrow$ Selection / filtering $\rightarrow$ Updated prompt(s)

Figure 4.1: Generic iterative loop of automatic prompt optimisation. Different APO methods instantiate different candidate-generation, evaluation, and selection operators.

4.3 Taxonomy of Available Prompt Optimisation Methods



Screenshot

Figure 1: Taxonomy of Automatic Prompt Optimization

Figure 4.2: Taxonomy of automatic prompt optimisation methods. Illustration adapted from [7].

Recent survey work on automatic prompt optimisation organizes existing methods into a common iterative pipeline consisting of seed prompt initialization, candidate generation, evaluation, candidate filtering, and repeated search steps [7]. Within this general structure, several algorithmic families have emerged.

4.3.1 Gradient-Based and Soft-Prompt Methods

The oldest line of work optimises prompts using direct gradient information. In *soft prompting*, the prompt is represented not as text tokens but as trainable continuous vectors prepended to the input embedding sequence [8, 9]. This makes the optimisation differentiable and allows standard gradient descent on a continuous prompt parameter $\mathbf{p}$:

$$\mathbf{p}_{k+1} = \mathbf{p}_k - \eta \nabla_{\mathbf{p}} \mathcal{L}(\mathbf{p}_k), \tag{4.11}$$

where $\eta > 0$ is a step size and $\mathcal{L}$ is a downstream task loss.

A related direction is *gradient-based hard prompt search*, where gradients are used to guide token substitution in a discrete prompt [10]. Conceptually, one seeks

$$\pi^\star \in \arg\max_{\pi \in \mathcal{P}_{\text{hard}}} J_t(\pi), \tag{4.12}$$

but gradient information is used to rank candidate token replacements. However, these methods require access to model internals or at least token-level gradients and therefore are generally not applicable to LLM-as-a-Service APIs.

4.3.2 Reinforcement-Learning Approaches

A second family formulates prompt search as a sequential decision problem. A policy generates or edits prompts and receives a scalar reward derived from downstream task performance. Let $q_\phi(\pi)$ be a prompt-generating policy parameterized by ϕ. The optimisation objective is typically written as

$$\max_{\phi} \ \mathbb{E}_{\pi \sim q_\phi}[R(\pi)], \tag{4.13}$$

where $R(\pi)$ is the reward assigned to prompt π. In the present classification setting, one may identify

$$R(\pi) = J_t(\pi). \tag{4.14}$$

The corresponding policy-gradient identity is

$$\nabla_\phi \mathbb{E}_{\pi \sim q_\phi}[R(\pi)] = \mathbb{E}_{\pi \sim q_\phi}\left[R(\pi) \nabla_\phi \log q_\phi(\pi)\right]. \tag{4.15}$$

Methods such as RLPrompt use reinforcement learning to optimise discrete prompts without requiring direct gradients of the target model [11]. This class is attractive because it is compatible with black-box evaluation, but in practice it is often highly sample-inefficient: a large number of rollouts is needed before the policy stabilises. In a provider-API setting with expensive repeated evaluation, this is a substantial limitation.

4.3.3 LLM-as-Optimizer and Textual-Reflection Methods

A third family replaces explicit policy learning with the reasoning capabilities of an LLM itself. Here a meta-model observes prompt candidates, task examples, errors, or historical scores, and proposes revised prompts in natural language. The general update has the form

$$\pi_{k+1} = \mathcal{M}_{\text{meta}}(\pi_k, \mathcal{E}_k, \mathcal{H}_k), \tag{4.16}$$

where $\mathcal{E}_k$ denotes task feedback and $\mathcal{H}_k$ optional search history.

Representative examples include:

- **APE**, which generates candidate instructions from demonstrations and selects the best-performing one [12];
- **ProTeGi**, which interprets failure analysis as a textual analogue of gradient descent and iteratively rewrites prompts based on natural-language critiques [13];
- **OPRO**, which conditions on previously evaluated prompts and their scores and asks an LLM to propose a better prompt directly [14].

These methods are black-box compatible and often much easier to deploy than RL-based methods. However, many of them are essentially greedy or near-greedy local search procedures. In practice, this can lead to over-specialisation: fixing one subset of errors may introduce new regressions on previously correct examples.

4.3.4 Evolutionary and Genetic Search

A fourth family treats prompts as a population of candidate solutions and borrows mutation, recombination, and selection operators from evolutionary computation. Instead of refining only one incumbent prompt, these methods maintain diversity and search multiple regions of the prompt space in parallel. Let $\mathcal{C}_k = \{\pi_k^{(1)}, \dots, \pi_k^{(M)}\}$ denote the current population. A generic evolutionary step can be written as

$$\widetilde{\mathcal{C}}_{k+1} = \text{MutateCross}(\mathcal{C}_k), \tag{4.17}$$

$$\mathcal{C}_{k+1} = \text{Select}(\mathcal{C}_k \cup \widetilde{\mathcal{C}}_{k+1}). \tag{4.18}$$

Examples include EvoPrompt and PromptBreeder [15, 16]. In general, evolutionary prompt optimisation is well suited to discrete non-convex search spaces because it does not rely on smoothness, differentiability, or a single local improvement trajectory.

This family is especially relevant for post-drift recovery, because it naturally supports exploration under non-stationarity and can preserve multiple partially successful prompt strategies at once.

4.3.5 Bayesian and Program-Level Optimisation

A fifth line of work optimises prompts at the level of complete LLM programs rather than isolated instructions. Methods such as MIPROv2 jointly optimise instructions and demonstrations for modular multi-stage systems, using Bayesian optimisation to navigate the candidate space [17]. Let $c \in \mathcal{C}$ denote one structured program configuration and let $\widehat{J}_k$ denote a surrogate model fitted after k evaluations. A Bayesian optimisation loop may be written as

$$\mathcal{S}_k = \{(c^{(j)}, J_t(c^{(j)}))\}_{j=1}^{k}, \tag{4.19}$$

$$\widehat{J}_k = \mathrm{FitSurrogate}(\mathcal{S}_k), \tag{4.20}$$

$$c^{(k+1)} = \arg\max_{c \in \mathcal{C}} a(c; \widehat{J}_k), \tag{4.21}$$

where $a(\cdot; \widehat{J}_k)$ is an acquisition function balancing exploration and exploitation.

This is an important baseline because it is explicitly designed for structured LLM pipelines and compound AI systems. However, Bayesian program optimisers typically focus on global aggregate scores. In settings where preserving already-correct behaviour is crucial, this aggregate perspective may be too coarse.

4.4 GEPA: Genetic–Pareto Evolutionary Prompt Optimisation

In this work, prompt adaptation is performed using the GEPA (Genetic–Pareto Evolutionary Prompt Optimisation) framework introduced by Agrawal et al. [18]. GEPA is a black-box prompt optimisation method for LLM-based systems, designed for settings in which optimisation must rely only on observable input–output behaviour rather than gradients or access to model internals. The method combines evolutionary search, natural-language reflection, and Pareto-based candidate selection. According to the original paper, GEPA begins from a base system, repeatedly proposes improved candidates under a rollout budget, and returns the candidate with the best aggregate validation performance. :contentReference[oaicite:0]index=0

4.4.1 Taxonomy Placement

Within the taxonomy of Automatic Prompt Optimisation methods, GEPA is best understood as a *population-based evolutionary black-box prompt optimiser with LLM-guided mutation*. Its main components are:

- population-based search over prompt candidates,
- reflective prompt mutation using language-model feedback,
- Pareto-based candidate filtering across task instances.

This places GEPA in the evolutionary family of prompt optimisation methods, while distinguishing it from purely greedy prompt-rewriting methods through its use of candidate pools and Pareto-based exploration.

4.4.2 Algorithmic Workflow

GEPA optimises a *pool* of candidate systems rather than a single prompt. Initially, the candidate pool contains only the base system:

$$P_0 = \{\Phi\},$$

where Φ denotes the initial prompt configuration with fixed model weights. The paper formalises the optimisation loop as follows: the training data are split into a feedback set D_{feedback} and a validation set D_{pareto}; the current candidate pool is maintained together with instance-level validation scores; and optimisation continues until the rollout budget is exhausted. :contentReference[oaicite:2]index=2

At iteration k, GEPA performs the following steps:

1. **Candidate selection.** A candidate Φ_k is sampled from the Pareto-filtered candidate pool rather than always choosing the globally best one.
2. **Module selection.** One module of the system is chosen for update. In the original algorithm, this is done according to a fixed policy such as round-robin. :contentReference[oaicite:4]index=4
3. **Minibatch rollout.** The selected candidate is executed on a minibatch sampled from D_{feedback}. This produces outputs, intermediate reasoning traces, and evaluation feedback. :contentReference[oaicite:5]index=5
4. **Reflective prompt update.** The current prompt, rollout traces, and textual feedback are given to a reflection language model, which proposes an updated prompt for the selected module. The paper's meta-prompt explicitly instructs the reflection model to infer task structure, extract domain-specific lessons from examples and feedback, and write revised instructions.
5. **Acceptance test.** The updated candidate Φ' is re-evaluated on the minibatch. If its average minibatch score improves over the parent, it is added to the candidate pool and then evaluated on D_{pareto}. Otherwise, it is discarded. :contentReference[oaicite:7]index=7

After the rollout budget is exhausted, GEPA returns the candidate with the best aggregate score on D_{pareto}.

4.4.3 Core Design Principles

GEPA is built around three design principles stated in the original paper: genetic prompt evolution, reflection using natural-language feedback, and Pareto-based candidate selection.

Black-box optimisation. GEPA does not require gradients, weight updates, or access to hidden states. Candidates are evaluated only through an external metric μ and the associated feedback function μ_f. This makes the method directly applicable to LLM-as-a-Service APIs. :contentReference[oaicite:10]index=10

Evolutionary population search. GEPA maintains a candidate pool P and repeatedly derives new candidates from existing ones. In the terminology of the paper, new candidates are obtained by reflective mutation or, in the GEPA+Merge variant, by crossover between lineages. Each candidate therefore represents one point in an evolving search tree of prompt configurations.

Reflective mutation. Unlike classical genetic algorithms that rely on random perturbations, GEPA performs *semantic* mutation. The algorithm records execution traces, reasoning, tool use, and textual evaluator feedback, and then uses a reflection language model to diagnose failures and propose a revised instruction. The paper describes this as implicit credit assignment at the module level.

Pareto-based selection. To avoid collapsing the search onto a single current best candidate, GEPA keeps candidates that are strong on different task instances. The paper's Pareto-based selection procedure first identifies the best candidate(s) for each instance, removes dominated candidates, and then samples among the remaining ones with probability proportional to how often they appear on this filtered frontier. :contentReference[oaicite:13]index=13

4.4.4 Summary

In short, GEPA is not just a prompt rewriter. It is a budget-constrained evolutionary search procedure over prompt candidates, where mutation is guided by natural-language reflection and candidate retention is governed by Pareto-style diversity preservation. This combination is precisely what gives GEPA its reported sample efficiency and makes it structurally different from greedy prompt editing methods.

4.5 Formal Description of GEPA

For the task considered in this thesis, GEPA can be formalised naturally on the fixed optimisation subset. Let

$$\mathcal{D}_{\text{opt}} = \{(x_1, y_1), \ldots, (x_n, y_n)\} \tag{4.22}$$

be the optimisation set, derived from the disagreement-based monitoring subset or a training partition thereof. For any candidate prompt π, define the per-instance correctness vector

$$\mathbf{s}(\pi) = \big(s_1(\pi), \ldots, s_n(\pi)\big) \in \{0,1\}^n, \tag{4.23}$$

where

$$s_i(\pi) = \mathbb{I}(f_t(x_i; \pi) = y_i)\,. \tag{4.24}$$

Thus prompt optimisation is no longer viewed only through the scalar objective $J_t(\pi)$, but through the full vector $\mathbf{s}(\pi)$.

4.5.1 Pareto Dominance

Definition 4.1 (Pareto dominance)
For two prompts $\pi^{(A)}$ and $\pi^{(B)}$, we say that $\pi^{(A)}$ Pareto-dominates $\pi^{(B)}$, written

$$\pi^{(A)} \succ \pi^{(B)}, \tag{4.25}$$

if

$$s_i(\pi^{(A)}) \geq s_i(\pi^{(B)}) \qquad \text{for all } i = 1, \ldots, n, \tag{4.26}$$

and

$$s_j(\pi^{(A)}) > s_j(\pi^{(B)}) \qquad \text{for at least one } j. \tag{4.27}$$

Hence, a dominating prompt is never worse on any instance and strictly better on at least one instance.

4.5.2 Population and Frontier

Let $\mathcal{C}_k$ denote the set of prompt candidates available after iteration k. The Pareto frontier at iteration k is

$$\mathcal{F}_k = \{\pi \in \mathcal{C}_k \mid \nexists \tilde{\pi} \in \mathcal{C}_k \text{ such that } \tilde{\pi} \succ \pi\}. \tag{4.28}$$

Only frontier candidates are considered non-dominated. This preserves prompt diversity by allowing different prompts to remain alive if they are best on different subsets of instances.

4.5.3 Reflective Mutation

Given a selected parent prompt π_k, the optimiser gathers a minibatch of task traces, error cases, and textual evaluation feedback. A reflection model then produces a revised prompt

$$\pi'_k = \mathcal{M}_{\text{refl}}(\pi_k, \mathcal{T}_k, \mathcal{F}_k^{\text{text}}), \tag{4.29}$$

where $\mathcal{T}_k$ denotes execution traces and $\mathcal{F}_k^{\text{text}}$ textual feedback signals. This update is *semantic*: the LLM rewrites the prompt using natural-language diagnosis rather than low-level token perturbation.

4.5.4 Selection Logic

After evaluation, the candidate pool evolves according to dominance and retention. Let the current candidate set be $\mathcal{C}_k$ and let $\widetilde{\pi}$ be a newly generated offspring prompt. Then the updated pool is

$$\mathcal{C}_{k+1} = \text{ParetoRetain}(\mathcal{C}_k \cup \{\widetilde{\pi}\}), \tag{4.30}$$

where

$$\text{ParetoRetain}(\mathcal{A}) = \left\{\pi \in \mathcal{A} \;\middle|\; \nexists \pi' \in \mathcal{A} \text{ with } \pi' \succ \pi\right\}. \tag{4.31}$$

Hence GEPA does not update by replacing one prompt with one better prompt. Instead, it updates the entire candidate population by preserving all non-dominated candidates. This prevents the optimisation from collapsing prematurely to a single prompt that may overfit one subset of examples while damaging another.

The full conceptual loop can be written as

$$\mathcal{F}_k = \mathrm{ParetoRetain}(\mathcal{C}_k), \tag{4.32}$$

$$\pi_k \sim \mathrm{Sample}(\mathcal{F}_k), \tag{4.33}$$

$$\widetilde{\pi}_k = \mathcal{M}_{\mathrm{refl}}(\pi_k, \mathcal{T}_k, \mathcal{F}_k^{\mathrm{text}}), \tag{4.34}$$

$$\mathcal{C}_{k+1} = \mathrm{ParetoRetain}(\mathcal{C}_k \cup \{\widetilde{\pi}_k\}). \tag{4.35}$$

The final selected prompt is the candidate with the strongest held-out performance among the evolved pool.

4.5.5 Regression and Recovery Counts

A central issue in post-drift adaptation is that a candidate prompt may fix some examples while breaking others. Since Chapter 3 measures stability through paired changes in correctness, it is natural to define prompt updates in the same language.

Let π_0 be the currently deployed prompt and let π be a candidate update. Define the number of newly introduced regressions by

$$R_{\mathrm{new}}(\pi \mid \pi_0) = \sum_{i=1}^{n} \mathbb{I}\big(s_i(\pi_0) = 1,\ s_i(\pi) = 0\big), \tag{4.36}$$

and the number of recovered cases by

$$G_{\mathrm{fix}}(\pi \mid \pi_0) = \sum_{i=1}^{n} \mathbb{I}\big(s_i(\pi_0) = 0,\ s_i(\pi) = 1\big). \tag{4.37}$$

A purely scalar objective such as

$$J_t(\pi) = \frac{1}{n} \sum_{i=1}^{n} s_i(\pi) \tag{4.38}$$

optimizes only the net balance $G_{\mathrm{fix}} - R_{\mathrm{new}}$ and does not explicitly control the distribution of these changes. By contrast, Pareto-based search preserves candidates whose per-instance behavior differs qualitatively, which is especially attractive when minimizing R_{new} is operationally as important as maximizing G_{fix}.

4.6 Why GEPA is the Appropriate Choice for This Setting

4.6.1 Compatibility with API Constraints

The thesis operates in a pure LLM-as-a-Service setting. Therefore, any method requiring gradients, embedding access, or weight-level intervention is infeasible. GEPA is fully black-box and thus immediately compatible with provider APIs.

4.6.2 Sample Efficiency Under Limited Budget

Prompt adaptation here is not a one-time offline exercise but a recovery mechanism that may need to be triggered repeatedly after drift events. Therefore, optimisation budget matters. Methods based on reinforcement learning are theoretically attractive but empirically expensive. By contrast, GEPA was explicitly designed for sample-efficient adaptation and, in reported benchmark results, achieved stronger performance than GRPO while using far fewer rollouts [18].

4.6.3 Suitability for Binary Classification on Hard Boundary Cases

The optimisation subset used in this thesis is intentionally built from disagreement examples near the decision boundary. These examples are precisely the cases where small prompt changes matter most and where regressions are easiest to introduce. A method that preserves multiple competing prompt strategies is therefore preferable to one that follows only a single greedy trajectory.

GEPA is especially suitable here because the disagreement subset is effectively a multi-objective environment: different prompt variants may solve different boundary cases. Pareto-based search is designed for exactly this structure.

4.7 Integration into the Monitoring Loop

Within the overall framework of the thesis, prompt optimisation is activated only after a statistically supported degradation event. The operational logic is:

1. monitor the deployed model on the fixed subset;
2. detect a significant degradation event using the paired drift rule from Chapter 3;
3. if degradation is confirmed, run prompt optimisation on an optimisation subset derived from the same task distribution;
4. compare the recovered prompt against the current deployment and against alternative models using the cost-aware decision rule.

Thus, prompt optimisation is not a substitute for monitoring. It is the *adaptive response* once monitoring has established that recovery is necessary.

Drift detection on monitoring subset $\rightarrow$ Trigger APO $\rightarrow$ Optimize prompt on $\mathcal{D}_{\text{opt}} \rightarrow$
Re-evaluate recovered prompt $\rightarrow$ Compare with fallback models using utility

Figure 4.3: Role of prompt optimisation inside the monitoring pipeline. APO is triggered only after a statistically supported degradation event.

5 System Implementation and Experimental Evaluation

This chapter is split deliberately into two halves. The first half describes the implemented monitoring system as an artifact: how datasets are stored, how evaluation runs are executed, and how traces are collected and exposed for analysis. The second half turns to the experiments performed with that system: how the baseline was established, how longitudinal monitoring was carried out, and how drift-response decisions are evaluated. This separation is important because the architecture is the means of the thesis, whereas the monitored runs and their interpretation are the empirical outcome.

Part I: System Architecture and Implementation

5.1 Overview and System Design

The implemented system has three persistent layers:

- **Dataset layer:** Langfuse stores the labelled reference benchmark and the fixed monitoring subset;
- **Execution layer:** local scripts and provider batch APIs execute benchmark and monitoring runs;
- **Trace layer:** ingested Langfuse traces store normalized predictions, scores, usage, and costs.

Figure 5.1 gives the high-level design. The key architectural decision is that the system separates one-time dataset construction from repeated monitoring. A manual full-benchmark run on $N = 1200$ labelled examples is used once to define the disagreement-based subset, whereas the recurring weekly loop operates only on the fixed $n = 300$ monitoring set. This keeps repeated evaluation inexpensive while preserving paired comparability across time.

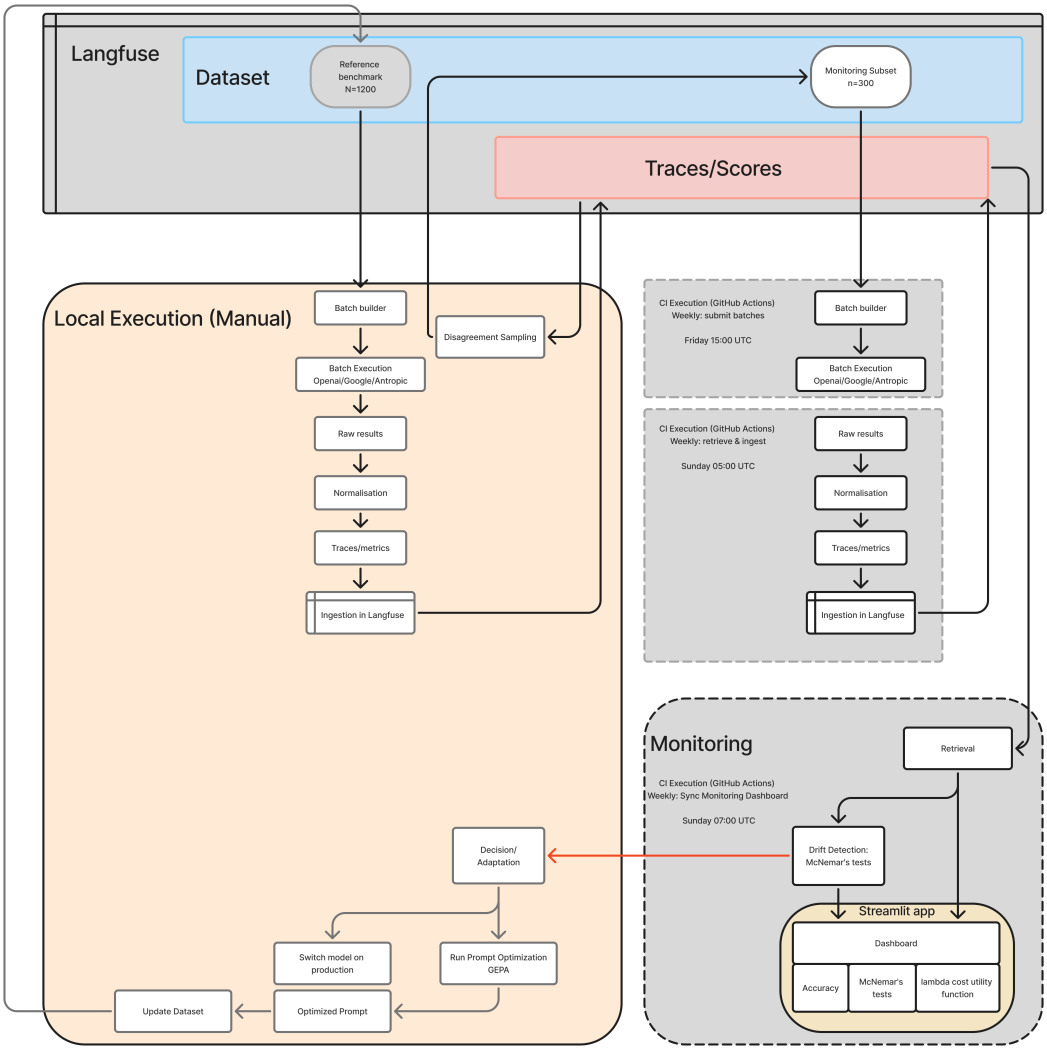


Figure 5.1: Overall system architecture: Langfuse stores datasets and ingested traces, batch execution runs either locally or via split GitHub Actions workflows, and monitoring is served through exported CSV artifacts synchronized to the Streamlit dashboard.

The same architecture therefore supports two run types:

- *Benchmark run:* a broader comparison on the full reference benchmark;
- *Monitoring run:* a recurring paired evaluation on the fixed 300-example subset.

This distinction is what prevents Chapter 5 from conflating system design with empirical monitoring. The benchmark run initializes the monitoring instrument; the weekly subset runs produce the longitudinal evidence used in the later sections.

5.2 Dataset Management and Trace Logging

The implementation uses two Langfuse datasets:

- **Reference benchmark** ($N = 1200$): Langfuse link;
- **Monitoring subset** ($n = 300$): Langfuse link.

Langfuse acts as the single source of truth for both dataset scales. Each row carries a stable `custom_id`, and that identifier is reused end-to-end across:

- batch submission,
- raw result files,
- ingested traces,
- CSV exports,
- statistical analyses.

Each dataset row includes the campaign description, post text, transcription, hashtags, media references, frame references where applicable, and the binary ground-truth label `campaign_relevant`. Reusing the same schema for both the $N = 1200$ benchmark and the $n = 300$ monitoring subset ensures that the smaller set is not a separate task definition but a frozen projection of the original benchmark onto its most informative cases.

The 300-example monitoring dataset is selected once from the full benchmark and then held fixed. This immutability is crucial: it makes per-example histories meaningful and ensures that paired McNemar comparisons are not confounded by changes in sample composition.

After each run, outputs are written back into Langfuse as traces. Each trace stores the normalized response, provider and model identifiers, token usage, monetary cost, and derived scores. Langfuse therefore serves two roles at once: it is both the registry of evaluation instances and the persistent log of how each monitored model behaved on each instance over time.

Figure 5.2 shows the concrete artifact flow from dataset storage to downstream analysis.

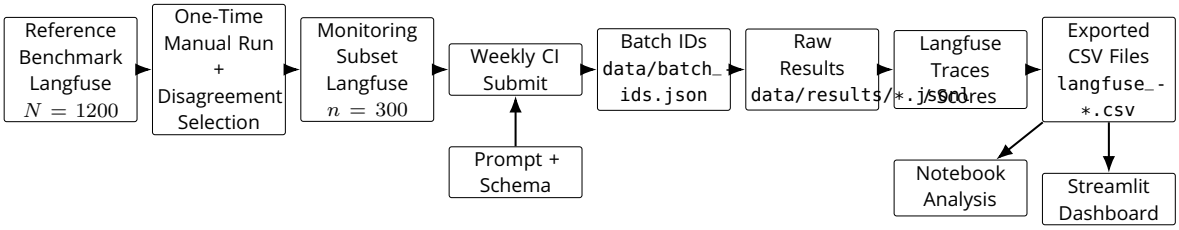


Figure 5.2: Concrete artifact flow: the full Langfuse benchmark is used once to define the fixed disagreement-based monitoring subset, weekly CI runs produce batch identifiers and raw result files, ingestion writes traces and scores back to Langfuse, and exported CSV files feed the dashboard and offline analysis.

5.3 Automated Evaluation Pipeline

The execution layer turns stored dataset rows into asynchronous provider-side batch jobs and later converts those jobs back into raw result files. Chronologically, one run consists of six steps:

1. load the target dataset split via the dataset loader;

2. combine each row with the shared prompt and response schema;
3. serialize the logical request through a provider-specific batch adapter;
4. submit the batch and persist the returned identifiers in `data/batch_ids.json`;
5. retrieve completed outputs into `data/results/`;
6. hand the raw outputs to the ingestion layer for normalization and trace creation.

The loader supports both Langfuse datasets and CSV files. The CSV path is operationally useful for the frozen monitoring subset because it guarantees that the identical 300 instances can be replayed even outside live dataset access.

The logical task is provider-invariant: every request asks for the same campaign-relevance classification under the same output schema. What changes is only the transport layer. The concrete set of monitored models, together with the prices used later for cost accounting, is defined centrally in `config/models.yaml` and summarized in Table 5.1.

Table 5.1: LLM models monitored in this study, grouped by provider.

Provider	Models
OpenAI	`gpt-5.4-2026-03-05`, `gpt-5.2`, `gpt-5.1`, `gpt-5`, `gpt-5-mini`, `gpt-5-nano`, `gpt-4.1-mini`, `gpt-4.1-nano`
Gemini	`gemini-2.5-flash`, `gemini-2.0-flash`
Claude	`claude-sonnet-4-6`, `claude-sonnet-4-5`, `claude-haiku-4-5`

The provider differences enter exactly at the serialization layer:

- **OpenAI:** `batch_openai.py` emits one JSONL line per example, already formatted as a call to `/v1/responses`;
- **Gemini:** `batch_gemini.py` rewrites the shared payload into Gemini's native batch format and replaces image URLs with references to files uploaded beforehand;
- **Claude:** `batch_claude.py` first stores a staging JSONL entry with a sanitized identifier and then translates the payload into Anthropic's Messages Batches format.

Gemini is the only provider that requires a dedicated pre-upload step for image-based inputs. An image-upload helper plus cache module therefore prevents repeated uploads of identical media across runs.

The structural differences are easiest to see in abbreviated examples:

Listing 5.1: Batch entry formats across providers.

```
=== OpenAI ===
{"custom_id":"...", "method":"POST", "url":"/v1/responses",
 "body":{
   "input":[...],
   "text":{"format":{"type":"json_schema"}}
 }}

=== Gemini ===
{"key":"...", "request":{
 "systemInstruction":{"parts":[{"text":"..."}]},
 "contents":[{"role":"user","parts":[
```

```
          {"text":"..."},
          {"fileData":{"fileUri":"..."}}
        ]}],
        "generationConfig":{
          "responseMimeType":"application/json",
          "responseSchema":{...}
        }}}

      === Claude (staging JSONL) ===
      {"custom_id":"...", "sanitized_id":"...",
       "body":{
         "input":[...],
         "text":{"format":{"type":"json_schema"}}
       }}
```

Across all three providers, the response is constrained to the same minimal schema:

$$\{\texttt{campaign\_relevant},\ \texttt{relevancy\_reasoning}\}.$$

This common schema is what keeps the monitoring stack provider-agnostic at the trace and scoring levels.

Routine execution is automated through a split GitHub Actions loop:

- submission workflows start new batches on Fridays at 15:00 UTC;
- retrieval workflows poll and ingest completed results on Sundays at 05:00 UTC;
- synchronization workflows export dashboard CSVs on Sundays at 07:00 UTC.

This split is not cosmetic. It reflects the asynchronous semantics of the provider batch APIs, where completion typically occurs within a roughly 24-hour window and retrieval must therefore be decoupled from submission. Figure 5.3 summarizes the implemented weekly loop.

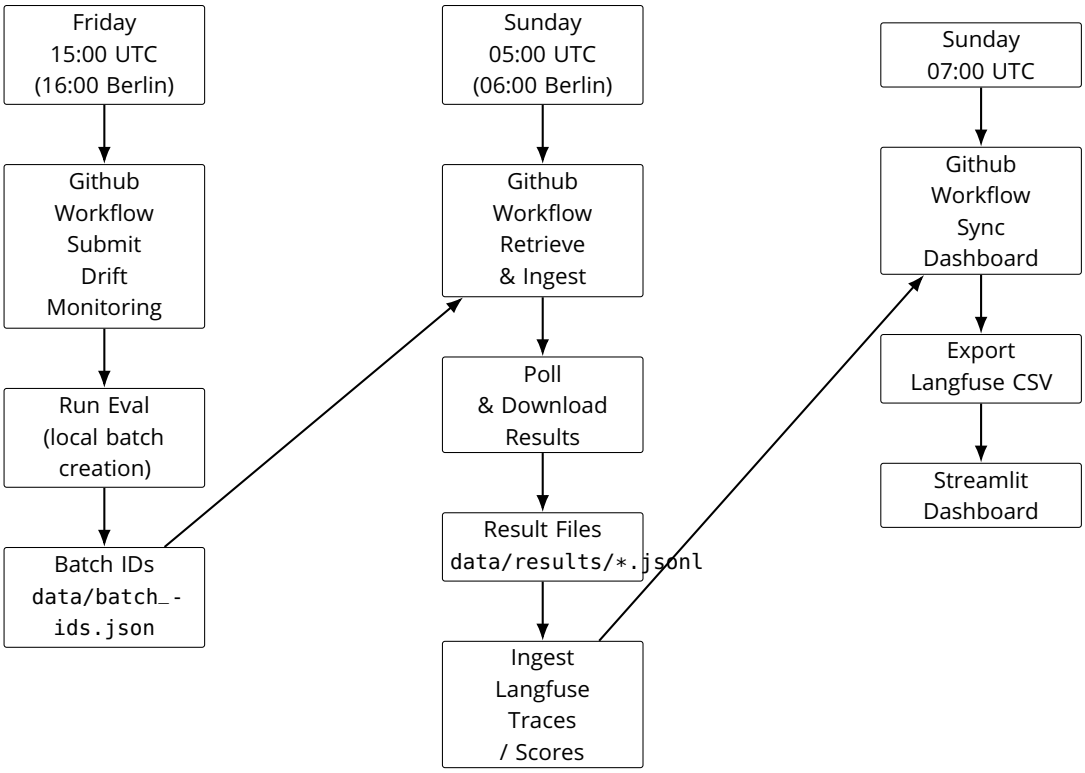


Figure 5.3: Automated weekly monitoring loop implemented via GitHub Actions. Batches are submitted on Friday afternoon, retrieved and ingested into Langfuse traces early Sunday morning, and the monitoring dashboard is refreshed from exported CSVs two hours later.

5.4 Metrics Collection and Dashboarding

Once raw batch outputs have been retrieved, the monitoring layer converts them into a common analytical representation. For each output line, ingestion performs five steps:

1. normalize the provider-specific response;
2. extract `campaign_relevant` and join it to the ground truth via `custom_id`;
3. compute token usage from the provider-specific usage fields;
4. convert token usage into USD cost via `config/models.yaml`;
5. write the resulting Langfuse trace together with derived scores and metadata.

The full batch-API price table for all monitored models is provided in Appendix A.1 as Table A.1.

Three trace-level scores are central for the later evaluation:

- `accuracy` $\in \{0, 1\}$, indicating per-example correctness;
- `error_type`, distinguishing true positives, false positives, true negatives, and false negatives;
- `cost_usd`, giving the realized dollar cost of the request.

Accuracy and cost are therefore first-class time-series variables rather than post hoc aggregates. This matters operationally because drift detection, model comparison, and error analysis are all derived from the same persisted traces.

The Streamlit dashboard does not query Langfuse directly. Instead, the synchronization workflow exports CSV snapshots into a dedicated dashboard directory, and the dashboard is rebuilt from these exported artifacts. This keeps the interface independent of live credentials and makes each weekly monitoring state reproducible.

The dashboard exposes four views used in the evaluation:

- per-model accuracy curves over time on the fixed monitoring subset;
- paired McNemar contingency tables for selected run pairs;
- confusion summaries derived from `error_type`;
- cost and utility views based on `cost_usd` and the deployment utility from Chapter 3.

Among these, the paired-drift and utility views are the operationally decisive ones because they turn raw traces into concrete actions: whether to keep the current setup, switch models, or attempt prompt recovery. Figure 5.4 shows the paired-drift interface, and Figure 5.5 shows the utility interface used for cost-aware comparison.

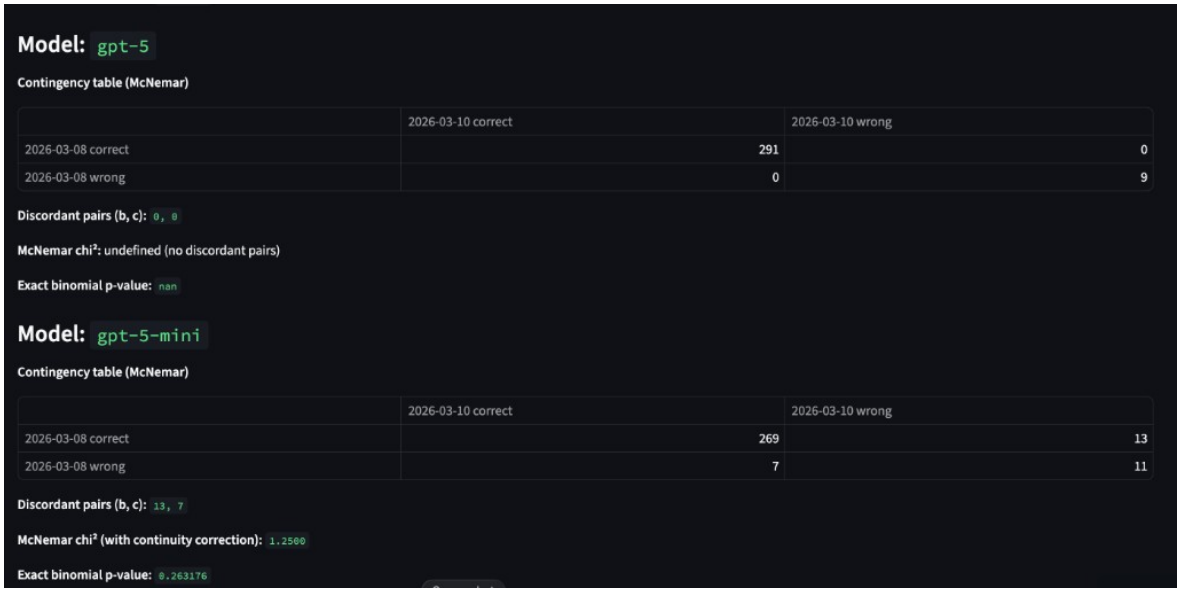


Figure 5.4: Streamlit visualisation of McNemar contingency tables and statistics for two monitored models. The top panel corresponds to `gpt-5`, where no discordant pairs occur, and the bottom panel to `gpt-5.1-mini`, where the discordant counts $(b, c) = (13, 7)$ yield a non-significant drift signal.

The same exported CSVs also support notebook-based analyses outside the dashboard. In practice, the dashboard is the operational surface of the monitoring loop, whereas notebooks are used for deeper inspection of drift events, error clusters, and prompt-recovery candidates.

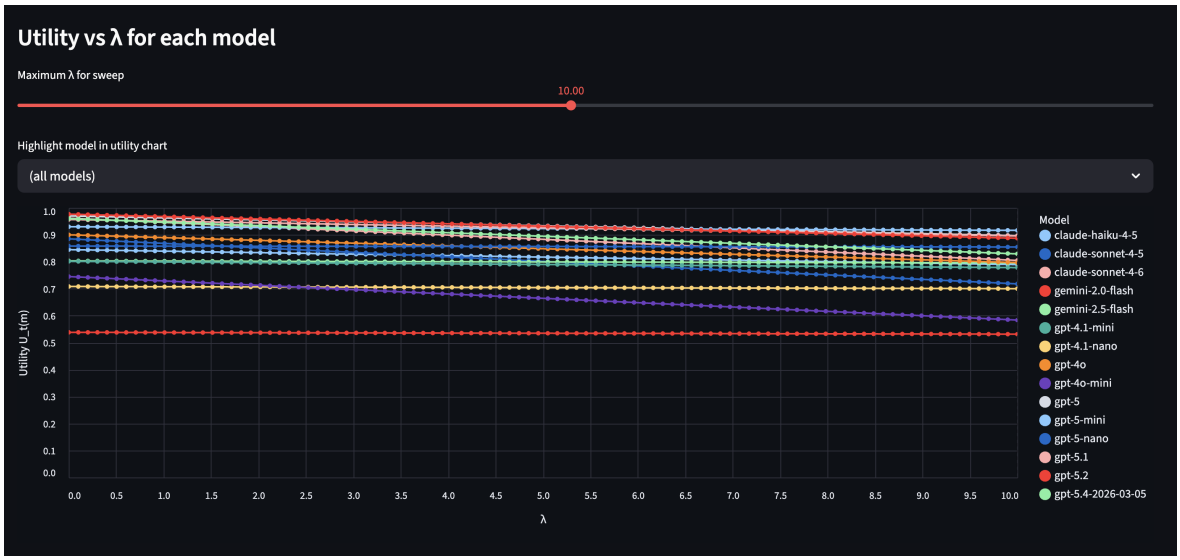


Figure 5.5: Overview of the deployment utility dashboard in the monitoring application. The top panel recalls the definition of $U_t(m)$ and exposes controls for the evaluation window, model set, and the cost-sensitivity parameter λ.

Part II: Experimental Evaluation and Results

5.5 Experimental Setup

The experiments instantiate the framework from Chapters 2–4 on two fixed dataset scales:

- the full reference benchmark with $N = 1200$ labelled examples;
- the disagreement-based monitoring subset with $n = 300$.

The monitored model roster is the one defined in `config/models.yaml`. Each run is versioned by dataset, prompt, and model configuration. Because the system records one trace per example, each run yields an aligned correctness vector that can be compared pairwise against its own baseline.

The monitored quantities are the same ones motivated in the methodology chapters:

- per-example correctness and aggregate accuracy;
- paired discordant counts (b, c) for McNemar testing;
- token usage and realized dollar cost;
- deployment utility $U_t(m)$ for corrective-action comparison.

Operationally, the experiments unfold in four phases. First, the full $N = 1200$ benchmark is run once to establish baseline behaviour and to construct the disagreement subset. Second, the fixed subset is used for longitudinal drift monitoring. Third, when alternative responses to degradation must be compared, models are ranked under the cost-aware utility from Chapter 3. Fourth, if a degradation event is statistically verified, GEPA is available as a bounded prompt-recovery mechanism.

The decision logic is:

1. compare a new run to its baseline for the same model on the same 300 examples;

2. raise a drift alert only if $b > c$ and $p < 0.05$ in the one-sided McNemar test;
3. if no drift is detected, keep the current setup;
4. if drift is detected, compare corrective actions under $U_t(m)$;
5. either switch model or run GEPA;
6. if GEPA is used, re-evaluate the recovered prompt on the same 300 examples.

The default deployment setting is $\lambda = 10$. GEPA runs under a fixed evaluation budget, and any recovery attempt is reported through c_{new}, b_{new}, Δaccuracy, and Δutility. Figure 5.6 summarizes this experimental logic.

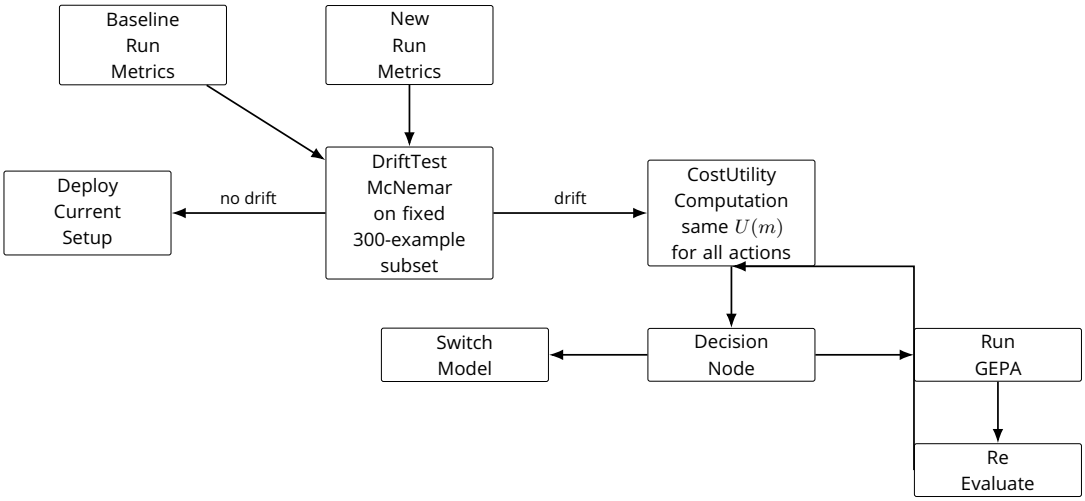


Figure 5.6: Decision logic after a monitoring run. Drift is tested on the fixed 300-example subset. If no statistically significant degradation is detected, the current setup is retained. If degradation is detected, corrective actions are compared under the same deployment utility $U(m)$; this may lead either to model switching or to prompt optimisation with subsequent re-evaluation.

5.6 Phase 1: Establishing the Baseline and Monitoring Subset

The first empirical phase is the one-time full-benchmark run on the $N = 1200$ reference dataset. Its purpose is not yet longitudinal drift detection, because there is no time axis at this stage. Instead, it establishes the initial per-model behaviour on the full benchmark and identifies those examples on which model decisions diverge. These disagreement cases are the operational signal used to construct the smaller monitoring subset described in Chapter 2.

This phase therefore produces two outputs. First, it yields a full benchmark record that can be reused for broad model comparison. Second, it produces the fixed $n = 300$ disagreement-based monitoring set that is stored as a separate Langfuse dataset and then frozen for all later paired analyses. The key result of this phase is not merely that a subset exists, but that it is anchored in observed cross-model uncertainty rather than arbitrary sampling.

Once created, the monitoring subset becomes the empirical backbone of the thesis. All later drift statistics, dashboard views, and adaptation decisions are computed on this fixed set. The full benchmark remains important as the initial reference frame, but the recurring evidence in the remainder of the chapter comes from the repeated reuse of the same 300 difficult cases.

5.7 Phase 2: Longitudinal Drift Monitoring

The second phase evaluates whether repeated runs on the fixed monitoring subset reveal meaningful temporal changes in model behaviour. Because the subset is held constant, any change in the paired counts (b, c) can be attributed to changed model predictions rather than to a changed sample.

Two monitored examples illustrate the range of outcomes. For `gpt-5`, the run pair from 8 March 2026 to 10 March 2026 yields the contingency table in Table 3.2. The discordant counts are $(b, c) = (0, 0)$, so no monitored instance changed correctness between the two dates. In operational terms, this is the strongest possible evidence of short-term stability on the subset.

For `gpt-5.1-mini`, the corresponding run pair in Table 3.3 yields $(b, c) = (13, 7)$. This means that 20 monitored cases changed status, with 13 regressions and 7 recoveries. Accuracy therefore decreased numerically, but the one-sided McNemar test does not cross the $\alpha = 0.05$ threshold. The result is an interpretable weak-degradation pattern: the model worsened on more cases than it improved on, but the evidence is not yet strong enough to trigger the thesis' alerting rule.

Figure 5.4 shows how these cases appear in the monitoring interface. The important empirical point is not just whether an alert fires. The paired view also preserves the structure of change: it distinguishes complete stability from mixed regressions and recoveries, and it makes visible when a decrease in accuracy is still statistically inconclusive.

5.8 Phase 3: Cost-Aware Model Selection

The third phase asks how a monitoring system should choose among candidate corrective actions once predictive quality and cost are both taken seriously. Because cost is logged per trace during ingestion, each monitored run can be evaluated under the deployment utility $U_t(m)$ introduced in Chapter 3 rather than by accuracy alone.

The effect of the cost-sensitivity parameter λ is structurally clear:

- at $\lambda = 0$, the ranking reduces to pure predictive performance;
- as λ increases, token-efficient models gain utility even if they are slightly less accurate;
- at the default $\lambda = 10$, the preferred action reflects an explicit compromise between performance and operating cost.

This matters because a drift response is not a binary technical fix. A more accurate fallback model may still be a poor deployment choice if the cost increase is disproportionate. Conversely, a cheaper model is only attractive if the drop in monitored performance is sufficiently small. Figure 5.5 shows the dashboard view used to inspect these trade-offs across date windows and model sets.

The result of this phase is therefore a decision surface rather than a single universally best model. The monitoring system turns model selection into an explicit function of observed behaviour and budget preference, which is exactly what is needed when comparing continued deployment, model switching, and prompt recovery under one common criterion.

5.9 Phase 4: Prompt Adaptation (GEPA)

The fourth phase addresses what happens after a verified degradation event. In this thesis, GEPA is not treated as a general prompt-improvement tool but as a conditional recovery mechanism that is invoked only when monitoring has already established harmful drift, that is, when $b > c$ and $p < 0.05$.

The implemented recovery workflow is:

1. start from the currently deployed prompt;
2. construct an optimization set from hard and disagreement cases;
3. run GEPA under a fixed evaluation budget;
4. select the recovered prompt from the evolved frontier;
5. re-evaluate that prompt on the same 300-example monitoring subset.

Recovery is then measured with the same quantities used in the rest of the chapter: c_{new}, b_{new}, Δaccuracy, and Δutility. This is what makes prompt recovery commensurable with model switching rather than a separate qualitative intervention.

Within the monitoring window documented here, however, no run shown in this chapter crosses the full alert threshold required to trigger GEPA as a production intervention. The empirical contribution of this phase is therefore the implemented and evaluation-ready recovery mechanism, not yet a completed live recovery case with before/after prompt text. Figure **??** shows this recovery path.

5.10 Error Analysis and Limitations

The qualitative error patterns are concentrated near the class boundary, but the two dominant failure modes are different in kind. False positives typically arise when the model overweights superficial lexical cues such as brand mentions, campaign slogans, or hashtags and infers campaign relevance from weak surface association alone. False negatives, by contrast, often arise when campaign relevance is implicit, visually grounded, or distributed across text and image context so that no single textual field states the relevance criterion explicitly.

This asymmetry matters for interpretation. False positives indicate overgeneralization from easy textual signals, whereas false negatives indicate missed contextual integration. In a monitoring setting, the latter are often harder to recover from with simple prompt wording because they can reflect genuinely multimodal ambiguity rather than a missing textual rule.

The main validity limits are:

- the 300-example subset is intentionally biased toward hard cases and is therefore a monitoring instrument, not an unbiased estimator of population accuracy;
- provider-side changes are opaque, so detected drift is observable only at the task-output level rather than attributable to internal model updates;
- cost may shift over time because of pricing changes or tokenization changes even when predictive behaviour remains stable;
- GEPA can overfit the monitored hard cases, which is why it is reserved for verified drift events and always re-evaluated on the same fixed subset.

Taken together, the results show that the implemented system can establish a hard-case monitoring subset, track paired temporal changes on that subset, and compare corrective actions under a common utility criterion. What the chapter does not yet provide is a fully observed end-to-end recovery episode in which drift is detected, GEPA is triggered, and the recovered prompt is deployed.

6 Degradation Case Study and Recovery Results

While the monitoring window in Chapter 4 did not yet contain a statistically significant degradation that crossed the full alert threshold, practical deployments will eventually encounter such an episode. This chapter specifies how those results are reported once a harmful change is detected. It is written so that a future degradation can be dropped into a fixed structure without further design choices.

6.1 Alerted Degradation Case

The starting point is the first monitoring run pair in which the one-sided McNemar test satisfies the thesis' alert criterion $b > c$ and $p < 0.05$ for the deployed model m^*. Let the corresponding monitoring dates be t_0 (reference) and t_1 (degraded). The alert is defined on the fixed 300-example subset, so any change in accuracy between t_0 and t_1 is attributable to changed model behaviour rather than sample noise.

For the alerted pair, the accuracy trajectory is reported as a short monitoring history, typically the last $k \in \{3, 4\}$ subset runs leading up to t_1. This is visualised as a univariate line plot of subset accuracy over time with the alerting point marked explicitly. The key information is when the degradation occurred relative to previous stability rather than only the single-step drop.

6.2 McNemar Test at the Trigger Point

The statistical evidence at the alert point is summarised in a dedicated McNemar contingency table for the pair (t_0, t_1). As in Chapter 4, the table reports the paired counts

	correct at t_1	incorrect at t_1
correct at t_0	a	b
incorrect at t_0	c	d

together with the discordant counts (b, c), the one-sided p-value, and the corresponding change in subset accuracy $\Delta\text{acc} = \text{acc}(t_1) - \text{acc}(t_0)$. This table is the canonical object that justifies the alert: it makes visible not only that accuracy decreased, but how many individual traces regressed or recovered.

6.3 Cost-Sensitive Utility Comparison

Because the monitoring system is cost-aware, degradation is evaluated not only in terms of raw accuracy but also under the deployment utility $U_t(m)$ introduced earlier. For the alerted period, utility is reported for both t_0 and t_1 as

$$U_t(m) = \text{acc}_t(m) - \lambda \cdot \text{cost}_t(m),$$

where λ is the current cost-sensitivity parameter. The chapter reports $\Delta U = U_{t_1}(m^*) - U_{t_0}(m^*)$ alongside Δacc so that degradation is visible both in accuracy and in utility space.

For operational decision-making, the same utility is also evaluated for candidate fallback models $m' \in \mathcal{M}$ on the monitoring subset at t_1. The results are presented as a short comparison table or bar chart over models, showing $\text{acc}_{t_1}(m')$, $\text{cost}_{t_1}(m')$, and $U_{t_1}(m')$ for one or more values of λ. This makes transparent whether a cheaper but slightly less accurate model overtakes the incumbent in terms of deployment utility once drift has occurred.

6.4 Prompt Optimisation Before and After

If GEPA is triggered for the alerted episode, the final part of the chapter reports the before/after prompt comparison on the same 300-example monitoring subset. Let P_{old} be the deployed prompt at t_1 and P_{new} the recovered prompt selected from the GEPA frontier. Both prompts are evaluated on the monitoring subset under the same model m^*, producing paired correctness counts and costs per trace.

The core quantities reported are:

- the change in subset accuracy $\Delta\text{acc}_{\text{prompt}} = \text{acc}(P_{\text{new}}) - \text{acc}(P_{\text{old}})$;
- the change in deployment utility $\Delta U_{\text{prompt}} = U(P_{\text{new}}) - U(P_{\text{old}})$ at the chosen λ;
- the McNemar discordant counts for the prompt pair $(P_{\text{old}}, P_{\text{new}})$, again with a one-sided test focused on detecting improvement.

These quantities are complemented by a short qualitative summary of typical recovered cases, but the main result is numerical: whether the recovered prompt restores or exceeds the pre-degradation accuracy without violating the cost budget encoded by λ. Together, the alerted degradation case, the utility comparison, and the prompt optimisation results form a complete end-to-end recovery narrative that can be instantiated for any future drift episode.

Appendix A: Supplementary Tables

A.1 Batch Pricing Table

Table A.1: Batch-API prices per one million tokens (USD) for all models in `config/models.yaml` as of 16 March 2026.

Model	Input price	Output price
`gpt-5.4-2026-03-05`	1.25	7.50
`gpt-5.4`	1.25	7.50
`gpt-5.2`	0.875	7.00
`gpt-5.2-pro`	10.50	84.00
`gpt-5.1`	0.625	5.00
`gpt-5`	0.625	5.00
`gpt-5-pro`	7.50	60.00
`gpt-5-mini`	0.125	1.00
`gpt-5-nano`	0.025	0.20
`gpt-4.1`	1.00	4.00
`gpt-4.1-mini`	0.20	0.80
`gpt-4.1-nano`	0.05	0.20
`gpt-4o-2024-05-13`	2.50	7.50
`gpt-4o`	1.25	5.00
`gpt-4o-mini`	0.075	0.30
`o1`	7.50	30.00
`o1-pro`	75.00	300.00
`o1-mini`	0.55	2.20
`o3`	1.00	4.00
`o3-pro`	10.00	40.00
`o3-mini`	0.55	2.20
`o3-deep-research`	5.00	20.00
`o4-mini`	0.55	2.20
`o4-mini-deep-research`	1.00	4.00
`gemini-3-flash-preview`	0.25	1.50
`gemini-2.5-flash`	0.15	1.25
`gemini-2.5-flash-preview-09-2025`	0.15	1.25
`gemini-2.0-flash`	0.05	0.20
`gemini-2.5-pro`	0.625	5.00
`claude-opus-4-6`	2.97	14.88
`claude-opus-4-5`	2.97	14.88
`claude-opus-4-1`	8.92	44.63
`claude-opus-4`	8.92	44.63
`claude-sonnet-4-6`	1.78	8.92
`claude-sonnet-4-5`	1.78	8.92
`claude-sonnet-4`	1.78	8.92
`claude-haiku-4-5`	0.59	2.97
`claude-3.5-haiku`	0.40	2.00

Bibliography

[1] L. Chen, M. Zaharia, and J. Zou, *How is ChatGPT's behavior changing over time?*, 2023. DOI: 10.48550/arXiv.2307.09009 arXiv: 2307.09009 [cs.CL].

[2] S. Tu, C. Li, et al., *ChatLog: Carefully evaluating the evolution of ChatGPT across time*, 2023. DOI: 10.48550/arXiv.2304.14106 arXiv: 2304.14106 [cs.CL].

[3] J. Echterhoff et al., *MUSCLE: A model update strategy for compatible LLM evolution*, 2024. DOI: 10.48550/arXiv.2407.09435 arXiv: 2407.09435 [cs.CL].

[4] L. Cayton, "Algorithms for manifold learning", University of California, San Diego, Tech. Rep. CS2008-0923, 2005.

[5] C. Fefferman, S. Mitter, and H. Narayanan, "Testing the manifold hypothesis", *Journal of the American Mathematical Society*, vol. 29, no. 4, pp. 983–1049, 2016.

[6] Y. Bengio, A. Courville, and P. Vincent, "Representation learning: A review and new perspectives", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, pp. 1798–1828, 2013.

[7] K. Ramnath et al., *A systematic survey of automatic prompt optimization techniques*, 2025. DOI: 10.48550/arXiv.2502.16923 arXiv: 2502.16923 [cs.CL].

[8] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning", in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 3045–3059. DOI: 10.18653/v1/2021.emnlp-main.243

[9] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation", in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 2021, pp. 4582–4597. DOI: 10.18653/v1/2021.acl-long.353

[10] T. Shin, Y. Razeghi, R. L. Logan IV, E. Wallace, and S. Singh, "AutoPrompt: Eliciting knowledge from language models with automatically generated prompts", in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 4222–4235. DOI: 10.18653/v1/2020.emnlp-main.346

[11] M. Deng et al., "RLPrompt: Optimizing discrete text prompts with reinforcement learning", in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 2022, pp. 3369–3391. DOI: 10.18653/v1/2022.emnlp-main.222

[12] Y. Zhou et al., "Large language models are human-level prompt engineers", in *The Eleventh International Conference on Learning Representations*, 2023.

[13] R. Pryzant, D. Iter, J. Li, Y. Lee, C. Zhu, and M. Zeng, "Automatic prompt optimization with "gradient descent" and beam search", in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 7957–7968. DOI: 10.18653/v1/2023.emnlp-main.494

[14] C. Yang et al., "Large language models as optimizers", in *The Twelfth International Conference on Learning Representations*, 2024.

[15] Q. Guo et al., "Connecting large language models with evolutionary algorithms yields powerful prompt optimizers", in *The Twelfth International Conference on Learning Representations*, 2024.

[16] C. Fernando, D. Banarse, H. Michalewski, S. Osindero, and T. Rocktäschel, "Promptbreeder: Self-referential self-improvement via prompt evolution", in *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[17] K. Opsahl-Ong et al., "Optimizing instructions and demonstrations for multi-stage language model programs", in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 9340–9366. DOI: 10.18653/v1/2024.emnlp-main.525

[18] L. A. Agrawal et al., *GEPA: Reflective prompt evolution can outperform reinforcement learning*, 2025. DOI: 10.48550/arXiv.2507.19457 arXiv: 2507.19457 [cs.CL].

Statutory Declaration in Lieu of an Oath

I – Georgii Burdin – do hereby declare in lieu of an oath that I have composed the presented work independently on my own and without any other resources than the ones given.

All thoughts taken directly or indirectly from external sources are correctly acknowledged.

This work has neither been previously submitted to another authority nor has it been published yet.

Mittweida, 17. March 2026	
Location, Date	Georgii Burdin